BaiZe Technical Report

***Abstract*—Large language models (LLMs) exhibit strong reasoning and tool-use skills, yet their computational footprint makes most agentic models impractical for edge, embedded, and cost-sensitive electronic design automation (EDA) deployments. We present BaiZe, a ≈2.2-billion-parameter agentic language model built *from scratch* on a fixed compute budget around a *hybrid* state-space backbone. BaiZe adopts a sequential hybrid architecture—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—whose linear-time decoding is well suited to inference-heavy, cost-sensitive agents, and is trained with the NVIDIA NeMo recipe on Megatron-Core. The architecture is chosen through a matched from-scratch comparison against a dense transformer counterpart, in which the hybrid attains lower loss and faster inference with fewer parameters at a modest training-throughput cost. We then run a staged, budget-constrained hyper-parameter search (S1–S5) on $8\times$H100 over the final hybrid backbone, which selects a stable learning rate of 1×10^{-3} under a warmup–stable–decay schedule with an $86{:}10{:}4$ annealing mixture, reaching a validation loss of $\mathbf{2.2054}$ after 20000 steps and 2.6739 ± 0.0469 across three seeds at 5000 steps. We release the complete recipe and training command to support the reproducible construction of efficient agentic small language models.**

***Index Terms*—small language model, agentic LLM, hybrid architecture, state-space model, budget-constrained search, efficient inference**

I. Introduction

Large language models have recently demonstrated agentic capabilities—planning, tool invocation, and long-horizon reasoning—that make them attractive cores for intelligent assistants spanning chip design, verification, and other domains of electronic design automation. However, state-of-the-art agent backbones are routinely 7 B parameters or larger, demanding high-end accelerators and memory footprints that are incompatible with edge, embedded, and cost-sensitive deployments where inference latency and energy are first-class constraints. The constraint is sharper than raw cost in the settings we target: proprietary design artifacts—layouts, schematics and review judgements—cannot leave the premises, so an agentic core must be served locally; and because an agentic session accumulates context, what dominates serving cost is not prompt throughput but the memory that grows with the session. For a hybrid backbone this is architectural: only the four attention layers carry a session-growing state, while the Mamba-2 layers hold a constant-size recurrent one.

Recent small language models (SLMs, < 2 B) have closed much of the gap with larger counterparts on single-turn benchmarks, but building such a model *from scratch* remains an under-documented engineering problem. Every choice—architecture, optimizer, learning-rate schedule, data mixture—interacts with every other, and a naive grid search is prohibitively expensive. Our goal is therefore twofold: (i) to construct, from scratch and on a strictly limited compute budget, an end-to-end multimodal agentic system—an efficient hybrid language core paired with a custom vision encoder, aligned into a semiconductor-domain agent—and (ii) to expose the concrete, reproducible decisions that lead to it.

a) BaiZe.: We present BaiZe, an end-to-end multimodal agentic system assembled from scratch across five stages: (i) pre-training a ≈2.2B-parameter hybrid (state-space + attention) LLM, (ii) post-training it for agentic capability, (iii) pre-training a custom vision encoder, (iv) aligning the two into a multimodal model, and (v) adapting the result to the semiconductor domain. Three design pillars organise our work:

- **Architecture.** A sequential hybrid language backbone—56 layers interleaving Mamba-2 state-space layers, four grouped-query-attention layers, and MLP layers—giving linear-time decoding for inference-heavy agents, paired with a from-scratch vision encoder.
- **Reduced-horizon proxy search.** We exploit short, cheap proxy runs to rank candidate hyper-parameters, so that the ranking discovered at small scale carries over to the full model without re-tuning at scale.
- **Budget-constrained stage-wise pipeline.** Each of the five stages is trained under a hard GPU-hour budget, with a staged search protocol (S1–S5) for the language backbone and a single frozen evaluation protocol shared by the later stages.

We hope BaiZe serves as a minimal, reproducible blueprint for building efficient agentic SLMs from scratch, and that our search methodology is of practical value to cost-bounded training efforts in the EDA and embedded-AI communities.

II. Background and Related Work

A. Small Language Models and Agentic Reasoning

Parameter-efficient SLMs (< 2 B) have recently narrowed the gap with larger models on mathematical and commonsense reasoning. MiniCPM [1] introduced scalable training strategies and domain-weighted data mixing for small models; Qwen [2] and Gemma [3] demonstrate that strong code- and math-enriched corpora can lift small models substantially. Most of these efforts, however, treat the final trained model as the deliverable and report comparatively little about the *search* that produced it. Complementarily, agentic instruction-tuning datasets (e.g., tool-use and multi-step reasoning corpora) [4] have been shown to elicit planning and tool-invocation behaviour, yet are rarely paired with a from-scratch pre-training recipe for the 1B regime.

B. Hybrid and State-Space Architectures

State-space models (SSMs) such as Mamba [5] and Mamba-2 [6] replace full attention with a recurrent scan update, of-

fering linear-time decoding that suits inference-bound deployments. Hybrid designs interleave a small number of attention layers among SSM/MLP layers—notably Nemotron-H [7] and Hymba [8]—to recover long-range context while retaining fast decode. Released Megatron-Core variants of these families are scarce below 2B, leaving their small-scale trade-offs underdocumented; our controlled 2B comparison (Section IV-A) addresses precisely this gap.

C. Multimodal Vision-Language Models

Vision-language models connect a vision encoder to a language model through a lightweight projector, enabling end-to-end visual understanding and instruction following. LLaVA [9] established visual instruction tuning—aligning a vision encoder to a frozen LLM via a linear projection followed by instruction fine-tuning—as a scalable recipe for open multimodal assistants. LLaVA-OneVision [10] consolidates this line into a single open model spanning single-image, multi-image, and video tasks through a staged curriculum: stage-1 projector pre-training, stage-1.5 mid-training, and stage-2 instruction tuning. BaiZe follows this curriculum in stage (iv) (Section VII), substituting a from-scratch vision encoder (stage iii) for the frozen off-the-shelf encoder.

D. LLMs for Electronic Design Automation

A growing line of work applies LLM agents to chip design tasks such as register-transfer-level generation, verification, and bug triage [11], [12]. These applications motivate a cheap, always-on, agentic multimodal core that can run per-team or per-engineer; our work supplies the efficient multimodal backbone such systems require.

III. Architecture and Pipeline Overview

BaiZe is an end-to-end multimodal agentic system assembled in five stages: (i) from-scratch pre-training of a hybrid language backbone, (ii) post-training of that backbone (instruction tuning and reinforcement learning to strengthen agentic capability), (iii) pre-training of a custom vision encoder, (iv) alignment of the two into a multimodal large language model, and (v) semiconductor-domain adaptation via reinforcement learning. This section fixes the shared scaffolding—the two backbones, the overall pipeline, and the global training environment—before each stage is detailed in its own section.

A. Language Backbone

The language backbone is a sequential hybrid model [6], [7]: 56 transformer layers interleave Mamba-2 state-space layers, four grouped-query-attention (GQA) layers, and MLP layers. The layer allocation follows the Nemotron-H sequential hybrid convention—four attention layers among the 56, the remainder Mamba-2 and MLP—as provided by Megatron-Core's Mamba-hybrid recipe, whose 56-layer pattern we reuse verbatim while halving only the hidden and FFN widths to reach the ≈2B scale. BaiZe is not a Nemotron-H model: it is an order of magnitude smaller, contains no mixture-of-experts layers, and is pre-trained from scratch on an independent corpus under a fixed GPU-hour budget. Each Mamba-2 layer uses 8 state-space groups; the four attention layers use 16 query heads and 4 query groups (GQA) with untied word embeddings. Crucially, only the four attention layers carry rotary position embeddings (RoPE); the 24 Mamba-2 state-space layers are position-agnostic, so the backbone's long-context extrapolation quality depends on just $4/56{=}7.1\%$ of layers, an asymmetry exploited in Section IV-I. The full configuration is listed in Table I; the model totals ≈2.22B parameters.

TABLE I
BaiZe language-backbone configuration (Mamba2-hybrid).

Hyper-parameter	Value
Transformer layers	56
Architecture	hybrid (Mamba-2 + GQA + MLP)
Attention layers	4 (interleaved)
Hidden size	2048
FFN hidden size	8192
Attention heads (Q)	16
Query groups (GQA)	4
Mamba-2 state-space groups	8
Vocabulary size	129,281
Sequence length	4094
Activation	SiLU / SwiGLU
Normalisation	RMSNorm
Precision	BF16
Parameters	≈2.22B

B. Vision Encoder

The vision encoder is pre-trained from scratch in stage (iii) and is selected from six iso-parameter candidates at the 500–600 M scale; Section VI reports the comparison and its outcome.

C. Pipeline and Global Environment

Table II summarises the five-stage pipeline. All stages run on 8×NVIDIA H100 GPUs (CUDA 12.8, PyTorch 2.8.0, Python 3.10); fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled where applicable, with `tensor_model_parallel_size = 1` and BF16 precision throughout.

TABLE II
BaiZe five-stage pipeline overview.

Stage	Task	Status	Section
(i)	LM pre-training	complete	IV
(ii)	Post-training (SFT, RLVR)	design	V
(iii)	Vision encoder	complete	VI
(iv)	MLLM alignment	design	VII
(v)	Domain adapt. (RLHF)	design	VIII

IV. LLM Pretraining

Stage (i) pre-trains the hybrid language backbone (Section III-A) from scratch under a fixed budget. This section covers the architecture-selection rationale, the training recipe, and the resulting measurements; the backbone configuration itself is given in Table I.

A. Architecture Selection Rationale

We adopt the hybrid backbone on three grounds. **(i) Inference efficiency.** State-space layers decode with constant-time recurrence per token, while the sparse attention layers retain long-range context; this directly targets the inference-heavy, cost-sensitive deployments that motivate BaiZe. **(ii) Controlled comparison.** A matched from-scratch comparison at the ≈2B scale between a dense transformer (MiniCPM5-2B) and Mamba2-hybrid—trained under identical data, seed, and GPU footprint—favours the hybrid, which converges to a lower loss with fewer parameters (≈2.22B vs. 2.51B, -11.6%) and decodes faster under our direct-drive harness, at a $\sim$8% training-throughput cost (Table IV). Across all five checkpoints from 500 to 5000 steps, the hybrid consistently attains lower loss than the dense baseline, with the gap widening to $\Delta\approx 0.30$ by step 5000—confirming the advantage is not an early-training artifact and comes with an 11.6% parameter reduction. **(iii) Feasibility.** Megatron-Core ships a Mamba-hybrid provider whose 56-layer interleaving pattern we reuse verbatim, halving only the hidden and FFN widths to reach the ≈2B scale, making the backbone directly executable under the NeMo recipe without custom kernels.

B. Training Framework and Reduced-Horizon Search

We train with the NVIDIA NeMo pre-training entry point, driving the Megatron-Core [13] `NVIDIAMambaHybridModelProvider2B` provider directly with `tensor_model_parallel_size` $= 1$. To keep the entire sweep within budget, we run all hyper-parameter searches at a reduced iteration count and treat the resulting short-horizon ranking as predictive of the full-model optimum. Fused kernels, selective activation recomputation, and cross-entropy loss fusion are enabled to raise throughput.

C. Learning-Rate Schedule

BaiZe trains under a warmup–stable–decay-style learning-rate schedule whose hyper-parameters—warmup length, stable learning rate, and the decay ratio—are themselves swept in Section IV-D (cosine decay is kept as an alternative). The remaining schedule and batch-size settings (global batch size, sequence length, packing) are held fixed across configurations and re-measured on the final backbone, with figures reported where applicable.

D. Budget-Constrained Search Protocol

Because a full grid is infeasible under a fixed GPU budget, we adopt a staged, single-variable-at-a-time protocol:

- **S1 (baseline).** Reproduce the Mamba2-hybrid reference configuration (stable LR 3×10^{-4}, WSD) to establish a loss/throughput baseline.
- **S2 (learning-rate sweep).** Sweep the stable learning rate over seven points from 2×10^{-4} to 1×10^{-3} with WSD held fixed.
- **S3 (schedule axes).** On the winning learning rate, perturb one variable at a time—schedule family (WSD vs. cosine), decay ratio, warmup length, and min_lr.
- **S4 (annealing data mix).** Ablate the annealing data blend (pure Ultra-FineWeb-L3 vs. adding code and math) under the winning configuration.
- **S5 (long run + seeds).** Retrain the winning configuration from scratch over a 20000-step horizon, plus a three-seed short-horizon reproducibility check.

At every stage only a single variable changes relative to the running best, isolating each factor's contribution while keeping total GPU-hours within budget. The architectural search space is validated up front: the hybrid (Mamba2-hybrid) and dense (MiniCPM5-2B) candidates are compared head-to-head from scratch (Section IV-A), which selects the hybrid backbone before the hyper-parameter search begins.

E. Data

Pre-training uses the `Ultra-FineWeb-L3` English subset (≈690 B tokens across 616 parquet shards, 618 GiB), streamed and packed to a sequence length of 4096, tokenised with the DeepSeek-V4.1-Flash tokenizer (vocabulary size 129,281). Complementary corpora—code (`UltraData-Code`, ≈411 B tokens, 1.22 TB), mathematics (`UltraData-Math`, ≈303 B tokens, 552 GB), and an agentic instruction-tuning set (`UltraData-SFT-Agent-2609`, 51 G)—are reserved for post-training (Section V). For the full-scale production run, the larger `Ultra-FineWeb` base corpus (English high-quality tier, ≈2.66 TB / 6,000 parquet shards, plus a Chinese subset of 256 shards / 324 GB) is being prepared to supersede L3 as the stable-phase main corpus; the L3 subset is retained for the annealing tail. Contamination control is enforced before any corpus enters training: a blacklist of 193,295 character-level 13-gram fingerprints (FNV-1a 64-bit hashed, NFKC-normalised) is built from all 536 tasks across six evaluation snapshots of EDA-Eval-PyAether, and every new data shard—including SFT corpora—is scanned for ≥ 0.80 n-gram overlap before release (Section V-D).

F. Data Mixture Proportion Search

Because the optimal web/code/math ratio is scale-dependent and a full-scale sweep is infeasible, we searched the three-way mixture at a proxy scale using Bayesian optimisation. A compact proxy model (d=128, L=14, ≈18.36 M parameters—two orders of magnitude smaller than the 2.2 B target and not intended as a performance predictor) was trained for each trial under Optuna's `TPESampler` with `MedianPruner`, one trial per GPU, global batch size 2048 at sequence length 4096. The objective was the average accuracy on eight common-sense `lm-eval` tasks (`arc_challenge`, `arc_easy`, `boolq`, `hellaswag`, `openbookqa`, `piqa`, `sciq`, `winogrande`), each evaluated with `--limit 500` subsampling.

Two rounds of 200 trials each were completed. In the first round, using final training loss as the objective, the Spearman rank correlation between loss-ranked and accuracy-ranked mixtures was $\rho = -0.43$, confirming that loss is a weak proxy for downstream ability. In the second round, using the eight-task average directly, the top-5 mixtures were retrained

and re-evaluated at the full `lm-eval` scale. Table III reports the results: the full-eval rankings were nearly inverted relative to the Bayesian-optimisation rankings (Spearman $\rho{=}-0.80$, $n{=}5$, $p{\approx}0.10$), and the spread across all five mixtures was only 0.383–0.390 (≈ 0.6 pp).

TABLE III
TOP-5 DATA MIXTURES FROM THE SECOND BAYESIAN-OPTIMISATION ROUND, RETRAINED AND EVALUATED AT FULL `LM-EVAL` SCALE. THE BO RANKING (COLUMN 5) IS NEARLY INVERTED RELATIVE TO THE FULL-EVAL RANKING (COLUMN 6), AND THE SCORE SPREAD IS ≈0.6 PP.

Trial	Web	Code	Math	BO	Full
23	0.941	0.109	0.010	**0.4155**	0.3846
38	0.895	0.100	0.010	0.4138	0.3835
118	0.874	0.071	0.055	0.4130	0.3877
199	0.931	0.098	0.010	0.4123	**0.3899**
33	0.940	0.086	0.010	0.4090	0.3897

At the proxy scale of 18.36 M parameters, data-mixture rankings were indistinguishable within 0.6 pp, leading us to adopt the 88:8:4 (web/code/math) prior from the literature rather than over-fitting to proxy-scale noise. The empirically optimal region was stable at web 0.87–0.94, code 0.06–0.12, math ≈0.01, consistent with the prior. Raising the discrimination floor—via a larger proxy, multiple seeds, or full-scale evaluation replacing the `--limit 500` subsampling—is left to future work.

G. Search Budget

The pre-training search is bound by a ≈92 GPU-hour plan on eight H100 GPUs (≈192 GPU-hours wall-clock available); ≈85 GPU-hours were consumed. All short-horizon sweeps (S1–S4) run 5000 steps at a global batch size of $8 \times 4094 \approx 32.7$K tokens/step (≈164M tokens, ≈4.0 GPU-hours each); the verification long run (S5) runs 20000 steps (≈655M tokens, ≈13.9 GPU-hours). Validation loss is reported at matched step counts so that comparisons across configurations remain apples-to-apples.

H. Results

The winning configuration is a stable learning rate of 1×10^{-3} under a warmup–stable–decay (WSD) schedule (warmup 250 steps, decay 500 steps, $min_lr{=}1\times10^{-5}$), with an 86:10:4 blend of Ultra-FineWeb-L3, code, and math data in the annealing tail. Over the 5000-step horizon the validation loss reaches $\mathbf{2.7647}$. The learning-rate sweep improves monotonically from 2.9037 (2×10^{-4}) to 2.7627 (1×10^{-3}); because 1×10^{-3} is also the grid maximum, that on its own leaves open whether the true optimum lies beyond the grid. Extending the grid upward closes the question: 1.5×10^{-3} gives 2.7752 and 2×10^{-3} gives 2.7889, so the sweep is *U-shaped* and 1×10^{-3} is an interior minimum rather than a clipped boundary. Each short-horizon ablation confirms a single-variable choice: WSD beats cosine (2.7679 vs. 2.8155), a 10% decay ratio beats 5% (2.7679 vs. 2.7900), a 250-step (5%) warmup beats a fixed 2000-step warmup (2.7679 vs. 2.7957), and $min_lr{=}1\times10^{-5}$ and 3×10^{-5} are statistically indistinguishable (2.7647 vs. 2.7679, $\Delta{\approx}0.003$ within run-to-run noise of $\sigma{\approx}0.035$ across $n{=}5$ seeds); we adopt $min_lr{=}1 \times 10^{-5}$ as default. Adding code and math to the annealing mix lowers the final loss monotonically (2.7621 pure L3 → 2.6793 L3+code → 2.6298 L3+code+math).

The 20000-step verification run reaches a final training loss of $\mathbf{2.2054}$, with the decay tail (steps 19500–20000) annealing the loss from $\sim$ 2.34 to 2.2054 (annealing gain $\sim$ 0.14; Figure 1). Across three seeds the 5000-step loss is 2.6739 ± 0.0469 (mean $\pm$ sample standard deviation, $n{=}3$; per-seed 2.6402, 2.6539, 2.7275).

Table IV contrasts the hybrid backbone against a dense transformer at the ≈2B scale, trained from scratch under matched data, seed, and GPU footprint; the hybrid offers fewer parameters and faster decoding at a modest training-throughput cost.

TABLE IV
ARCHITECTURE-LEVEL COMPARISON MOTIVATING THE HYBRID CHOICE (DENSE = MINICPM5-2B, HYBRID = BAIZE; MEASURED FROM SCRATCH UNDER MATCHED DATA, SEED, AND GPU FOOTPRINT, SAME NODE, SAME CONFIGURATION). TRAINING THROUGHPUT WAS RE-MEASURED ON A SINGLE 8×H100 NODE AFTER AN INITIAL CROSS-NODE MEASUREMENT WAS FOUND TO OVERSTATE THE GAP. THE DECODE GAP IS MEASURED UNDER THE MEGATRON-CORE DIRECT-DRIVE HARNESS WITHOUT CUDA GRAPHS, WHERE THE DENSE BASELINE IS CPU-LAUNCH-BOUND, AND THEREFORE NARROWS UNDER PRODUCTION INFERENCE STACKS (CF. TABLE VI).

Metric	Dense	Hybrid	Edge
Parameters	2.51B	**2.22B**	hybrid (−11.6%)
Final loss (1k steps)	4.81	**4.68**	hybrid (−0.12)
Training tok/s	**90K**	83K	dense (+8%)
Prefill tok/s	21.4K	**23.7K**	hybrid (+10.5%)
Decode tok/s	2.2	**22.8**	hybrid (+10.3×)

I. Long-Context Capability and Inference Cost

The hybrid backbone's long-context behaviour was characterised along two axes: perplexity extrapolation and inference throughput under a production serving stack.

Perplexity extrapolation. With adjusted base frequency (ABF, $\theta_{\text{rope}}{=}10^6$) applied to the four attention layers, the 20B-token checkpoint sustains near-constant perplexity from 4K to 1M tokens without further training: PPL ranges from 51.5 (64K) to 55.4 (1M), a change of only +7.6% over a 256× context increase (Table V). This stability arises because only 4 of 56 layers carry RoPE position encoding; the 24 Mamba-2 state-space layers have no position dependency and therefore incur no out-of-distribution penalty at long range.

Inference throughput. Under the SGLang production serving stack (CUDA graphs, PagedAttention, FlashInfer backend) with warm-up, the hybrid outperforms the dense baseline at long context: at 128K the hybrid achieves 2.8× higher prefill throughput and 1.6× higher decode throughput; at 256K the gaps widen to 4.3× and 1.9× respectively (Table VI). The dense model cannot serve beyond 512K on a single 80 GB H100 (KV-cache pool cap ≈455K tokens at 30% memory

TABLE V
TRAINING-FREE LONG-CONTEXT PERPLEXITY OF THE MAMBA2-HYBRID (20B-TOKEN CHECKPOINT, ABF θ=10^6, b=1).

Context	PPL	Prefill (s)	Peak VRAM
4K	83.2	—	—
64K	51.5	—	—
128K	53.3	10.7	10.9 GB
256K	53.0	13.8	14.3 GB
512K	55.8	53.0	19.7 GB
1M	55.4	172.4	34.7 GB

fraction), whereas the hybrid serves through 1M with near-constant ≈26 GB VRAM, as the Mamba-2 state grows in $O(1)$ for 52 of 56 layers.

TABLE VI
LONG-CONTEXT INFERENCE COST COMPARISON (SGLANG, WARM-UP CORRECTED, b=1, GEN_LEN=64, 80 GB H100). DENSE ENTRIES MARKED "—" ARE OUT-OF-MEMORY.

	Prefill (tok/s)		Decode (tok/s)	
Ctx	Hybrid	Dense	Hybrid	Dense
128K	39 377	14 051	311	193
256K	32 381	7 532	276	144
512K	23 634	—	220	—
1M	15 530	—	159	—

FP8 training feasibility. A 1000-step A/B comparison between BF16 and delayed-FP8 precision (FP8 with delayed scaling, M=65 536) shows no persistent loss divergence: the end-of-run loss differs by only 0.19% (2.5429 vs. 2.5477), and a single transient spike at step 700 recovers within ∼60 steps. FP8 delivers a 19.4% throughput gain (233K vs. 195K tok/s) and saves 5.6 GB per GPU. We therefore adopt delayed-FP8 as the precision for the full pre-training run.

Production training throughput. At the production configuration (TP=1, DP=8, micro-batch 2, seq 4094, BF16), the hybrid backbone sustains 249 040 tok/s over a 1100-step run on 8×H100, corresponding to ≈600 TFLOP/s/GPU.

Table VII reports the hyper-parameter search outcomes at a matched horizon; the searched axes target the hybrid training recipe (stable learning rate, warmup length, decay ratio, and schedule family).

TABLE VII
HYPER-PARAMETER AND DATA-MIX SEARCH OUTCOMES AT A MATCHED 5000-STEP (S5: 20000-STEP) HORIZON.

ID	Change vs. best	Val. loss	Outcome
S1-01	baseline (lr=3×10^{-4}, WSD)	2.849	baseline
S2-07	stable LR = 1 × 10^{-3}	**2.7627**	best LR
S3-02	schedule family: cosine	2.8155	WSD retains
S3-03	decay ratio 5%	2.7900	10% retains
S3-04	warmup 2000 steps	2.7957	250 retains
S4-03	anneal L3+code+math 86:10:4	**2.6298**	best mix
S5-01	winner, 20000 steps	**2.2054**	**winner**

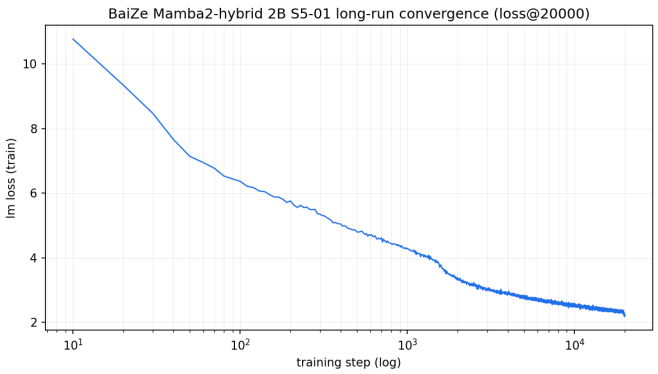


Fig. 1. Training-loss curve of the winning Mamba2-hybrid configuration over the 20000-step verification run (seed 1234); the decay tail (steps 19500–20000) anneals the loss from ∼ 2.34 to 2.2054.

V. LLM POST-TRAINING

Stage (ii) adapts the pre-trained backbone to EDA scripting, and it is the stage that carries BaiZe's domain claim: the pre-training corpus of Section IV-E is general web, code, and mathematics text and contains no EDA artifacts, so the domain prior must be introduced here. We organise the stage as three steps—*domain annealing*, *supervised fine-tuning*, and *reinforcement learning*—all evaluated under a single frozen protocol (Section V-D).

A. Domain Annealing

The cheapest lever on the domain prior is to continue pre-training on EDA text before any behavioural tuning. We reuse the warmup–stable–decay machinery of Stage (i) and anneal on a corpus assembled from PyAether and SKILL API references, EDA tool documentation, and open HDL sources, blended with the general mixture at a controlled ratio. Because the step changes only the data distribution, we contrast it against an SFT-only control to isolate its contribution from the behavioural tuning that follows.

B. Supervised Fine-Tuning

SFT teaches the generate–execute–refine loop that the deployment target requires. The corpus combines the agentic instruction set (`UltraData-SFT-Agent-2609`, 51 G across 50 shards) and a general instruction set (`UltraData-SFT-2605`, 319 G across 1,504 JSONL files, verified byte-identical to the HuggingFace manifest) with PyAether-specific material synthesised from API documentation. We ablate the data mixture (agentic / code / mathematics / PyAether), the presence of interleaved tool-call traces, full fine-tuning versus LoRA, and the sequence length: agentic traces—code, tool returns, and successive refinements—routinely exceed the 4096-token window of Table I, so window extension is treated as a first-class variable rather than a fixed setting.

C. Reinforcement Learning with Verifiable Rewards

An execution sandbox turns correctness into a verifiable signal, so we optimise with group-relative policy optimisation and no value network. Table VIII separates the reward variants we study. A binary signal is the simplest baseline; partial credit over executable attempts addresses the sparsity of pass/fail; a tool-call penalty aligns the policy with the harness cost model; and an API-legality term penalises hallucinated symbols. Because the reward is computed from assertions a policy could in principle target directly, we keep the harness anti-cheat hook enabled in every run and report a cheat rate beside every accuracy figure.

TABLE VIII
REWARD VARIANTS STUDIED IN STAGE (II).

ID	Reward
R1	binary pass/fail
R2	executable + assertion pass ratio
R3	R2 + tool-call penalty
R4	R3 + API-legality term

D. Evaluation Protocol

All post-training variants are scored on **EDA-Eval-PyAether** [14], a benchmark of 158 real-world EDA scripting tasks. Each task pairs a natural-language prompt and a function signature with assertion-based test code, and is scored by execution in the vendor sandbox: a task passes when the generated code runs without error and every assertion holds. We follow the benchmark's own protocol so that our numbers are comparable to published ones: five independent runs with reshuffled task order and fresh context, reporting mean Pass@1 and its run-to-run standard deviation, with a 2500-second per-task timeout and a single non-restartable trace per task.

Two controls make the comparison interpretable. First, *contamination*: no task, assertion, or reference solution from the benchmark enters any training split. A blacklist of 193,295 character-level 13-gram fingerprints (FNV-1a 64-bit hashed, NFKC-normalised, with 8-gram fallback for short texts) is built from all 536 tasks across six evaluation snapshots of EDA-Eval-PyAether; every training shard—including SFT corpora—is scanned for ≥ 0.80 n-gram overlap before release, with MinHash(k=128) tracing for source attribution. PyAether training material is drawn only from API documentation, and we isolate it by symbol so that benchmark targets cannot leak. Second, *harness parity*: only the backbone is varied. The retrieval stack, sandbox, anti-cheat hook, and agent scaffold are held byte-identical to the published configuration, because the harness supplies most of the observed accuracy and a variant harness would confound the model's contribution with the scaffold's.

Results are reported as **[TBD]**.

VI. VISION ENCODER PRETRAINING

Stage (iii) pre-trains a custom vision encoder from scratch for the multimodal stage (Section VII). This section presents the architecture-selection rationale, the pre-training recipe, and the resulting measurements.

A. Architecture Selection

We compare six candidate vision encoders at an iso-parameter 500–600 M scale, all *randomly initialized* (no upstream ImageNet/CLIP/SigLIP weights) so that only the architecture body is contrasted. All candidates are trained under a unified stack: `open_clip` 3.2.0 loss modules with a *frozen* CLIP-ViT-L/14-336 text tower (768-d, context 77), the CLIP InfoNCE (softmax-contrastive) objective, and the Stanford GPIC `short` caption split ($\approx$ 2.77 M pairs, 0% truncation), at resolution 224 / patch 16 / bf16, learning rate 3×10^{-3} (AdamW, warmup 20), seed 1234, batch 64×8 (i.e. 512 in-batch negatives). The six candidates are:

- **OpenVision2** — a pure-attention ViT (width 1024, depth 30), 505.2 M.
- **MambaEye** — a pure visual state-space model (Vim-style bidirectional SSM), 535.2 M.
- **MoE-ViE** — a sparse Mixture-of-Experts encoder (222.2 M active per token), 505.5 M total.
- **DeepEncoderV2** — an Attention+SSM hybrid, 517.2 M.
- **AIMv2** — a ViT-GELU variant, 505.5 M.
- **FastViTHD** — a conv-stem + hierarchical conv/attention hybrid, 507.8 M.

The last two are *iso-parameter adaptations* of the AIMv2 and FastViTHD designs to this scale, not the official released models. Table IX reports the measured parameter counts, the training loss at 3000 steps, training throughput, and—crucially—two downstream metrics on ImageNet-1k measured on the *frozen* trunk: zero-shot top-1/top-5 and linear-probe top-1.

TABLE IX
VISION-ENCODER COMPARISON (FROM-SCRATCH, UNIFIED `open_clip` LOSS MODULES + CLIP INFONCE + FROZEN CLIP-VIT-L/14-336 TEXT TOWER; STANFORD GPIC `short`; 3000 STEPS; LR 3×10^{-3}; 512 IN-BATCH NEGATIVES). DOWNSTREAM METRICS ARE MEASURED ON THE *frozen* TRUNK (IMAGENET-1K SELF-SPLIT, 50K VAL IMAGES), SO THEY PROBE THE LEARNED REPRESENTATION RATHER THAN AN ATTACHED CLASSIFIER; ZERO-SHOT TOP-1 CHANCE IS 0.10%. **ZS T1/T5** = ZERO-SHOT TOP-1/TOP-5 (%); **LP T1** = LINEAR-PROBE TOP-1 (%). † RUN TERMINATED EARLY BY THE COLLAPSE CIRCUIT-BREAKER (SEE TEXT).

Arch.	Params	Loss@3k	img/s	zs t1/t5	lp t1
OpenVision2	505.2M	**4.865**	3052	**0.95/3.64**	**1.14**
MambaEye†	535.2M	6.251	950	0.10/0.50	0.10
MoE-ViE	505.5M	5.558	1226	0.35/1.53	0.28
DeepEncoderV2†	517.2M	6.251	1865	0.10/0.50	0.10
AIMv2 (adapt.)	505.5M	5.637	**3393**	0.38/1.61	0.25
FastViTHD (adapt.)	507.8M	5.668	2097	0.39/1.57	0.27

Finding. OpenVision2 ranks first on every quality metric—lowest loss, most dispersed features, and the best ImageNet-1k zero-shot and linear-probe accuracy (Table IX). The pure-attention family (plus MoE or a conv stem) trains normally: AIMv2 is the fastest to train but does not convert speed into quality, and neither hybrid beats a plain attention ViT at equal parameter count.

An architecture-specific failure mode. The two candidates that contain state-space sub-structure—MambaEye (pure SSM) and DeepEncoderV2 (attn+SSM)—*exactly* collapse: their loss pins at $6.25 \approx \ln 512$, the entropy of an unlearned 512-way InfoNCE problem, with a same-tower feature cosine of 1.0000 and ImageNet-1k at exactly chance (0.10%). The text tower, data, objective, batch size and seed are identical across the six runs, so the only variable is the vision architecture; under this contrastive recipe the collapse is tied to the SSM sub-structure rather than to the recipe. We report it as a negative result, with its boundary stated: an earlier sweep under a different objective trained MambaEye without collapsing, so this is a property of the (architecture, objective) pair rather than evidence that state-space vision encoders are unusable.

Methodological note. An earlier sweep at $lr{=}10^{-3}$ pinned every ablation cell at an identical loss, because the cosine schedule collapsed the learning rate to $\approx 10^{-5}$ before the run ended. We therefore re-ran all configurations at $lr{=}3 \times 10^{-3}$; sub-1k-step comparisons, and any 3000-step comparison at $lr{=}10^{-3}$, are uninformative, and every number in this section uses the corrected rate.

B. Scaling Behaviour

Holding the winning architecture fixed, we ask how far the representation improves with data, and how tower width trades against data at a fixed sample budget. Stage-1 trains three widths for 30000 steps on the same $\approx$ 18.5 M-pair corpus (CC12M + Amshaker) under the recipe above: linear-probe top-1 falls monotonically with width—6.10% (w512, 126.8 M), 3.63% (w768, 284.5 M), 1.03% (w1024, 505.0 M). At this data scale the *smaller* tower is therefore the more sample-efficient one. Stage-2 then trains w512 for 108,000 steps (55.3 M samples) and evaluates every 10,000-step checkpoint on the frozen trunk (Table X): linear-probe top-1 rises from 3.96% at 5.1 M samples to a peak of 7.70% at 51.2 M.

TABLE X
DATA-SIDE SCALING OF THE SELECTED VISION TOWER (OPENVISION2, W512 = 126.8 M, FROZEN TRUNK, IMAGENET-1K LINEAR PROBE AT 10,000-STEP CHECKPOINTS).

Samples (M)	5.1	10.2	20.5	30.7	41.0	51.2	55.3
lp top-1	3.96	4.39	6.46	7.35	7.30	**7.70**	7.40

A power law $acc = 0.251 - 0.864\, N^{-0.090}$ fits the eleven checkpoints ($R^2 \approx 0.94$) and implies an asymptote of 25.1% *for the pure contrastive (InfoNCE) objective*: even with unlimited data this from-scratch contrastive route saturates near 25% linear-probe top-1, and reaching 20% would take on the order of 6×10^{10} samples under the more favourable log-linear extrapolation—roughly three orders of magnitude beyond the $\approx 1.2 \times 10^8$ unique image–text pairs available locally. A complementary five-point model-size sweep (widths 384–1024 at matched samples) finds the marginal effect of width to be negative throughout (≈ -2.2 to -2.4 linear-probe points per doubling of parameters) with no saturation point: the best configuration observed is the smallest tower tried (71.5 M, 7.99% at 15.4 M samples). We initially read this as the central limitation of the vision stage—the binding constraint is data volume and objective function, not architecture. The next subsection revisits that reading: the ceiling is specific to the contrastive objective, and switching to a caption-*independent* dense objective while retaining the contrastive term breaks it.

C. Breaking the Contrastive Ceiling with Dense Supervision

The 25.1% asymptote above is tied to a single global scalar signal per image–text pair (the InfoNCE loss). A body of recent work argues that *dense per-patch* supervision can be more sample-efficient than global contrastive learning: AIMv2 [15] reports "denser supervision compared to discriminative objectives" from a masked-patch generative head, SigLIP 2 [16] augments sigmoid contrastive loss with captioning and masked-prediction heads, and OpenVision 2 [17] removes the text encoder entirely in favour of a purely generative objective. Motivated by these findings, we test whether augmenting the contrastive loss with a caption-*independent* dense term can break the 25.1% ceiling without additional data.

a) AIMv2-style objective.: We add a masked-patch reconstruction head to the selected tower: 60% of patches are masked and their raw pixels are reconstructed by a lightweight predictor (0.66 M params) via an MSE loss, while the InfoNCE contrastive term is retained at equal weight ($\lambda_{\text{patch}} = 1.0$). The dense term is *caption-independent*—it supervises raw pixels, not text tokens—so it does not depend on caption quality or truncation. This is a self-developed adaptation of the AIMv2 design philosophy, *not* an official reproduction: the official AIMv2 [15] uses prefix-LM attention, $\alpha \approx 0.4$ loss weighting, and $\sim$ 12 B training pairs, none of which we replicate here.

b) Result: the ceiling is broken.: Under the *identical* architecture (OpenVision2, w512), data ($\approx$ 18.5 M-pair corpus, CC12M + Amshaker), and optimisation schedule as the scaling study above, the AIMv2-style objective produces a monotonically increasing linear-probe trajectory that exceeds the contrastive baseline at every checkpoint (Table XI): lp top-1 rises from 11.50% at 5.1 M samples to **19.76**% at 55.3 M—already above the 7.40% contrastive baseline at the same sample budget by +12.4 pp, and the gap *widens* monotonically from +7.5 pp at 10.2 M to +12.4 pp at 55.3 M. A power-law fit $acc = a - b\, N^{-c}$ yields $R^2 = 0.91$; the exponent $c = 0.040$ is much smaller than the contrastive-only fit ($c = 0.090$), so the curve saturates far more slowly and no asymptote can be identified within the observed range. A subsequent extension to 176 M samples (3 epochs over the full 58.8 M-pair corpus, 344 k steps) reaches 20.27%, still rising with no saturation signal; a 35-checkpoint refit gives $R^2 = 0.94$, but the power-law asymptote hits its upper bound ($a \to 1.0$) and the trajectory is non-monotonic (7 of 34 checkpoints decrease, attributable to resume transients), so the precise ceiling remains indeterminate.

TABLE XI
AIMV2-STYLE OBJECTIVE VS. CONTRASTIVE BASELINE (OPENVISION2, W512 = 126.8 M, FROZEN TRUNK, IMAGENET-1K LINEAR PROBE AT 10,000-STEP CHECKPOINTS). THE DENSE OBJECTIVE EXCEEDS THE CONTRASTIVE BASELINE AT EVERY CHECKPOINT, WITH THE GAP *widening* AS DATA GROWS. Δ = LP_{AIMv2} - $\text{LP}_{\text{InfoNCE}}$ (PERCENTAGE POINTS).

Samples (M)	5.1	10.2	20.5	30.7	41.0	51.2	55.3
InfoNCE lp	3.96	4.39	6.46	7.35	7.30	7.70	7.40
AIMv2-style lp	11.50	12.73	15.33	16.67	—	18.84	**19.76**
Δ (pp)	+7.5	+8.3	+8.9	+9.3	—	+11.1	+12.4

c) Causal decomposition: both terms are necessary.: A four-arm ablation at 15.4 M samples localises the effect:

- **Arm (i)** InfoNCE baseline: 6.08%.
- **Arm (iv)** CoCa (contrastive + *caption-dependent* generative CE): 0.47%—the caption-CE term collapses the frozen trunk, confirming that caption-*dependent* generative supervision is harmful in this regime.
- **Arm (vi-B)** AIMv2-style without the contrastive term (pure masked-patch AR): 6.23% (+0.15 pp over baseline—no breakthrough).
- **Arm (vi-A)** AIMv2-style with both terms: 12.08% (+6.00 pp—breakthrough).

The breakthrough therefore depends on the *interaction* of the contrastive anchor and the caption-independent dense term: neither alone suffices. Removing the contrastive term (arm vi-B) eliminates the breakthrough, and replacing the caption-independent term with a caption-dependent one (arm iv) actively collapses the representation. This is consistent with MAE [18], where pure masked-image modelling yields lower linear-probe accuracy than contrastive methods, and with AIMv2 [15] and SigLIP 2 [16], which retain a contrastive or sigmoid-contrastive signal alongside the dense head.

d) Boundary.: All AIMv2-style results are from a self-developed adaptation (not the official AIMv2), trained on ≤ 58.8 M unique pairs (vs. ~ 12 B for the official model [15]). The precise asymptote of the dense route remains indeterminate: a 35-checkpoint scaling refit over 5–176 M samples gives $R^2 = 0.94$, but the power-law parameter hits its upper bound ($a \to 1.0$) and the trajectory is non-monotonic (7 of 34 checkpoints decrease, attributable to resume transients), so we report it honestly without forcing an extrapolation. Nonetheless, 176 M samples show no saturation signal—the dense route reaches 20.27%, a level the contrastive route would need on the order of 10^{10} samples to attain—confirming that the 25.1% ceiling is specific to the contrastive objective. The full-data mix (GPIC + CC12M + Amshaker, 58.8 M pairs) is less sample-efficient at matched N than the CC12M+Amshaker-only corpus (16.80% vs. 19.76% at ≈ 55 M samples), likely because GPIC long and tag captions are truncated at the 77-token context; the caption-*independent* patch-MSE term is unaffected, and the dense route still exceeds the contrastive baseline by a wide margin (+9.4 to +12.4 pp at $\approx$ 55 M samples, depending on data mix). The four-arm decomposition is conducted at a single sample budget (15.4 M); the interaction effect (both terms necessary) is confirmed at larger scale, where the full AIMv2-style objective continues to exceed the contrastive baseline by a wide margin through 176 M samples, while removing the contrastive term (arm vi-B) yields no breakthrough at any scale tested.

D. Configuration and Ablations

Holding the winner (OpenVision2) fixed, we sweep the core degrees of freedom single-variable-at-a-time (3000 steps each, at $lr{=}3 \times 10^{-3}$—the rate at which the schedule does not collapse, see the note above): resolution/patch (Table XII), objective function (Table XIII), and learning rate, plus a multi-seed robustness check and a held-out retrieval proxy.

TABLE XII
RESOLUTION/PATCH ABLATION ON OPENVISION2 UNDER THE CORRECTED OBJECTIVE (RUN ON THE LLAVA `IMAGENET/EN` 500K ARM, HENCE A DIFFERENT LOSS SCALE FROM TABLE IX, WHICH USES GPIC), ALL RUNS 3000 STEPS. 224/16 IS BEST ON BOTH AXES: LOWEST LOSS AND HIGHEST THROUGHPUT; THE COARSEST GRID RETAINS ONLY $\approx 56\%$ OF THAT THROUGHPUT, SO ALL LATER STAGES USE 224/16.

res/patch	**Val. loss**
224/16	**3.0977**
336/14	3.1909
448/14	3.2711

TABLE XIII
OBJECTIVE FUNCTION AND LEARNING RATE ON OPENVISION2, ALL RUNS 3000 STEPS. THE TWO OBJECTIVES PRODUCE LOSSES ON DIFFERENT SCALES AND ARE *not* COMPARABLE; THE ONLY COMMENSURABLE QUANTITY IS THROUGHPUT, WHERE THEY TIE TO WITHIN 1%. LEARNING RATE, BY CONTRAST, IS DECISIVE: THE ORIGINAL ANCHOR COLLAPSED THE SCHEDULE TO ITS FLOOR.

Variant	**Loss**	**img/s**	**Note**
SigLIP	4.4562	1645	sigmoid
CLIP InfoNCE	3.2704	1657	softmax; scale differs
lr 10^{-3}	4.9962	1651	schedule collapsed
lr $3{\times}10^{-3}$	4.4562	1645	selected
lr $5{\times}10^{-3}$	4.4542	—	—

a) Mask-ratio ablation.: The dense patch-MSE term in the AIMv2-style objective masks a fraction of patches and reconstructs their raw pixels. We sweep the mask ratio $\{0.3, 0.5, 0.6, 0.75, 0.9\}$ at 30000 steps on the same w512 / CC12M+Amshaker / InfoNCE+patch-MSE (1:1) recipe, with the ratio 0.6 serving as the baseline (it reproduces the arm vi-A setting). Table XIV shows an inverted-U: 0.6 is optimal (13.49%), and both lower and higher ratios are worse, with the steeper falloff on the high side ($0.9 \to 11.53\%$, $\Delta{=}{-}1.96$ pp). None of the alternatives exceeds the +1.5 pp pre-registered threshold, so 0.6 is retained. The inverted-U is consistent with the MAE literature: too little masking starves the reconstruction signal, while too much masking leaves the predictor with insufficient context.

TABLE XIV
MASK-RATIO ABLATION ON THE AIMV2-STYLE OBJECTIVE (OPENVISION2, W512, 30000 STEPS, CC12M+AMSHAKER, INFONCE+PATCH-MSE 1:1, FROZEN-TRUNK IMAGENET-1K LINEAR PROBE). THE INVERTED-U PEAKS AT 0.6. Δ IS VS. THE 0.6 BASELINE.

Mask ratio	0.3	0.5	**0.6**	0.75	0.9
lp top-1	12.69	11.91	**13.49**	12.49	11.53
Δ (pp)	−0.80	−1.58	—	−1.00	−1.96

b) Contrast-to-patch weight-ratio ablation.: With the mask ratio fixed at 0.6, we sweep the relative weight of the InfoNCE contrastive term to the patch-MSE dense term: $\{1{:}2, 1{:}0.5, 0.5{:}1, 2{:}1\}$, against the 1:1 baseline. Table XV shows no arm breaks through the $+1.5$ pp threshold. The best arm (0.5:1, 13.80%, $\Delta{=}+0.31$) is within the noise band, but two arms are significantly worse: 2:1 (contrast-heavy, 10.21%, $\Delta{=}-3.28$) and 1:0.5 (patch-starved, 10.22%, $\Delta{=}-3.27$). The asymmetry is informative: increasing the patch weight (1:2) is nearly neutral ($\Delta{=}-0.36$), while increasing the contrastive weight (2:1) is strongly harmful—the contrastive term is the "critical" signal that must not be diluted, whereas the dense term is a "safe redundancy" that tolerates over-weighting.

TABLE XV
CONTRAST-TO-PATCH WEIGHT-RATIO ABLATION (SAME RECIPE AS TABLE XIV, MASK RATIO 0.6, 30000 STEPS). 1:1 IS THE BASELINE. NO ARM EXCEEDS THE +1.5 PP THRESHOLD; CONTRAST-HEAVY AND PATCH-STARVED ARMS ARE SIGNIFICANTLY WORSE.

contrast:patch	1:2	1:0.5	0.5:1	2:1
lp top-1	13.13	10.22	**13.80**	10.21
Δ (pp)	−0.36	−3.27	+0.31	−3.28

c) Evaluation.: Domain task metrics are not yet available for this stage, so we adopt the standard frozen-trunk protocol: ImageNet-1k zero-shot (class prototypes synthesised from the frozen CLIP text tower) and a linear probe on the frozen trunk, over a deterministic self-split of the ImageNet-1k training set (50 images per class for the probe and 50 for validation, disjoint from every corpus used for training). A retrieval proxy (R@1 on a held-out 5K subset of `laioncn/EN`) was degenerate (R@1=0.0002, exactly chance); we therefore use ImageNet-1k, on which architectures separate (2.5–9.5× chance vs. exactly chance for collapsed ones). Resolution is not a free variable in this comparison: $224/16$ is best on both loss and throughput (Table XII). Stage (iv) revisits the visual-token count for detail-dense layout and schematic views, where the reasoning of this section—that a 196-token view is dominated by coarse semantics—does not transfer.

E. Causal Autoregressive Paradigm Exploration

The AIMv2-style objective above uses a *bidirectional* vision trunk with random masking, which differs from the official AIMv2 [15], where the vision encoder uses *causal* (prefix-LM) attention and the objective is a pure autoregressive (AR) next-patch prediction plus a text-AR caption loss, with *no* contrastive term. The official model trains on ~ 12 B image–text pairs with long LLaMA-3 synthetic captions. We test whether the pure-AR paradigm is viable at our data scale ($\approx$ 7.7 M pairs, short alt-text captions) and whether the absence of a contrastive term causes the collapse observed in earlier experiments.

a) Setup.: Three arms are compared on the same OpenVision2 w512 architecture, data (CC12M+Amshaker), and 30000-step schedule:

- **Arm A** (baseline): bidirectional attention + InfoNCE + masked-patch-MSE (λ_{patch}=1.0, mask 0.6) → lp=13.49%.
- **Arm B** (official-style pure AR): causal (prefix-LM) attention + caption-AR loss + 0.4×pixel loss, *no* InfoNCE.
- **Arm B-hybrid**: causal attention + 1.0×InfoNCE + caption-AR loss + 0.4×pixel loss.

We monitor a collapse diagnostic C_1 (same-tower off-diagonal feature cosine; $C_1{\to}1$ means all features collapse to a single direction) and a gap guard C_2 (diagonal vs. off-diagonal attention entropy).

b) Result: pure AR collapses, hybrid survives but underperforms.: Arm B collapses within 600 steps: C_1 rises from 0.86 (step 300) to 0.97 (step 600), triggering the circuit-breaker at 2% of training. The loss is still decreasing (caption loss $6.77 \to 5.08$), so the collapse is a representation failure, not an optimisation failure—the AR next-patch task admits a trivial solution that collapses all features. This contrasts with a bidirectional model trained *without* contrastive loss ($C_1{\approx}0.43$, no collapse), showing that *causal* attention is more collapse-prone than bidirectional when the contrastive anchor is removed.

Adding the InfoNCE term (Arm B-hybrid) prevents collapse entirely: C_1 oscillates between 0.30 and 0.55 throughout 30000 steps, with a positive C_2 gap and no fuse trigger. However, the linear-probe accuracy is dramatically lower: lp=1.37% at step 30000, compared with 13.49% for the bidirectional baseline—a drop of 12.1 pp. Zero-shot accuracy is essentially at chance (0.37% vs. 0.10% for 1000 classes). The causal constraint—each patch attends only to its prefix—restricts information flow, and at our data scale ($\sim$ 7.7 M pairs with short captions) this restriction severely degrades representation quality.

TABLE XVI
CAUSAL AR PARADIGM EXPLORATION (OPENVISION2, W512, 30000 STEPS, CC12M+AMSHAKER). ARM A IS THE BIDIRECTIONAL BASELINE; ARM B IS THE OFFICIAL-STYLE PURE AR; ARM B-HYBRID ADDS INFONCE TO THE CAUSAL AR PARADIGM. C_1 IS THE SAME-TOWER FEATURE COSINE ($\to 1$ = COLLAPSE).

Arm	**Attn.**	**InfoNCE**	C_1	**lp top-1**	**Status**
Arm A (baseline)	bidirectional	✓	~ 0.28	**13.49%**	healthy
Arm B (pure AR)	causal	—	0.97@600	—	*collapsed*
Arm B-hybrid	causal	✓	~ 0.48	1.37%	no collapse, lo

c) Interpretation.: Two findings emerge: (i) The contrastive term is *necessary* to prevent collapse in the causal

AR paradigm at our data scale: without it, the AR objective admits a trivial collapse solution. (ii) The causal AR paradigm *cannot match* the bidirectional paradigm's representation quality at our scale, even with the contrastive anchor: prefix-only attention limits each patch to its causal context, degrading linear-probe accuracy. The official AIMv2 avoids this with $\sim 12\,\mathrm{B}$ pairs and long LLaMA-3 captions; our $\sim 7.7\,\mathrm{M}$ short-caption pairs cannot sustain the causal AR route. We retain the bidirectional + InfoNCE + masked-patch-MSE recipe.

d) Optimizer ablation: Muon.: Replacing AdamW with the Muon optimizer (Newton–Schulz orthogonalized momentum) under the same AIMv2-style objective causes contrastive collapse at every tested learning rate ($C_1 \to 0.96$ within $4\,200$ steps at $\mathrm{lr}=10^{-4}$; $2\,100$ steps at 5×10^{-4}), whereas the AdamW baseline trains stably for $187\,000$ steps ($C_1 \approx 0.1$–0.5). Reducing the learning rate delays but does not prevent collapse, indicating a structural incompatibility rather than a tuning failure. This is consistent with recent work showing that Muon's uniform spectral whitening—which drives all momentum singular values toward 1—amplifies noisy tail directions in low-rank or low-SNR gradient regimes [19], suggesting that the optimizer's applicability boundary is determined by gradient spectral shape rather than loss-function type.

VII. MLLM Alignment

Stage (iv) connects the vision encoder of Section VI to the language backbone through a lightweight projector, following the staged curriculum established for open multimodal assistants: projector pre-training, mid-training, and instruction tuning. The design is motivated by the observation that the dominant EDA artifacts are visual—layout and schematic views account for 70.2% of the tasks in the evaluation benchmark [14]—so a multimodal core can be scored on the same execution-based protocol as the text-only one.

a) Task construction.: We build three task families, each chosen because it admits an objective ground truth. (i) *image→script*: given a rendered layout or schematic view, produce the PyAether or SKILL script that recreates it; supervision is available because executing a script yields its own reference rendering. (ii) *script→image*: predict the geometry that a given script produces, again self-labelled by execution. (iii) *layout VQA*: answer attribute questions—layer identity, device type, spacing and rule violations—labelled by domain experts. The first family is the strongest of the three: its output is a script, so it is scored by the same assertion sandbox used in stage (ii), which gives the multimodal and text-only stages a common metric instead of two incommensurable ones.

b) Curriculum and ablations.: We run stage-1 projector pre-training on caption-style pairs, stage-1.5 mid-training on the three families above, and stage-2 instruction tuning on the full mixture. The ablations vary the projector (MLP versus query-based), whether the vision tower is frozen or updated, and the number of visual tokens per image. The last is of particular interest. The architecture study of Section VI selected $224/16$, where an image becomes only 196 tokens—a regime in which the training objective is dominated by coarse semantics; whether that setting transfers to detail-dense views is precisely what stage (iv) tests. Layout and schematic views are the opposite: they are detail-dense line drawings whose meaning lives in fine geometry. We therefore treat resolution and visual-token count as first-order variables here rather than inheriting the earlier null result, which we read as a property of the proxy task and not of the encoder.

Results are reported as **[TBD]**.

VIII. Semiconductor Domain Adaptation

Stage (v) completes the pipeline by adapting the multimodal model to proprietary design practice. It differs from stage (ii) in two respects, which we state explicitly because otherwise the two stages would coincide: the reward is no longer verifiable but *learned from human judgement*, and the target behaviour is no longer general API usage but a team's internal conventions—naming rules, layer policies, review checklists—which no public benchmark measures.

a) Method.: We collect pairwise judgements from in-house design reviews over the stage-(iv) task families, fit a domain reward model on them, and optimise the aligned model against that model under a KL-regularised objective. Because preference data is far scarcer than the verifiable signal of stage (ii), we initialise from the stage-(iv) checkpoint and constrain the update rather than training the objective from the pre-trained model.

b) Why a small model.: The rationale for a $\approx$2.2B core is strongest here, and it is a constraint rather than a preference. Proprietary layouts and proprietary preference data cannot leave the premises, so the choice is not between a large hosted model and a small local one—it is between a small local model and no model at all. Serving cost then dominates, and for an agentic session the relevant cost is not throughput on a short prompt but memory that grows with accumulated context: in the hybrid backbone of Section III-A, the four attention layers are the only component whose serving state grows with the session, while the Mamba-2 layers hold a constant-size recurrent state.

c) Evaluation.: We evaluate on held-out review items using expert agreement, and report the result beside the execution-based numbers of stage (ii) so that verifiable and preference-based adaptation can be compared on the same system rather than in separate accounts.

Results are reported as **[TBD]**.

IX. Conclusion

We presented BaiZe, an end-to-end multimodal agentic system: a $\approx$2.2B-parameter hybrid (state-space + attention) language model built from scratch, post-trained for agentic capability, paired with a custom vision encoder, aligned into a multimodal model, and adapted to the semiconductor domain. Through a controlled from-scratch comparison, we selected a sequential Mamba2-hybrid backbone over a dense transformer for its lower loss-per-parameter and substantially faster inference, then applied a staged, budget-constrained hyperparameter search (S1–S5) on the final backbone. Stages (ii)

to (v) are specified here as a protocol and evaluated on a common execution-based benchmark [14]; their measurements are reported as **[TBD]**. The complete recipe and a single reproducible command encapsulate the entire construction under the NVIDIA NeMo recipe.

The immediate next step is agentic fine-tuning with the `UltraData-SFT-Agent` corpus toward a deployable agent core for EDA and embedded applications. We hope this blueprint lowers the barrier for cost-bounded teams building their own efficient agentic SLMs from scratch.

REFERENCES

[1] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao *et al.*, "Minicpm: Unveiling the potential of small language models with scalable training strategies," *arXiv preprint arXiv:2404.06395*, 2024.

[2] A. Yang, B. Li, B. Zhang *et al.*, "Qwen3 technical report," 2025.

[3] G. Team, "Gemma 3 technical report," 2025.

[4] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang *et al.*, "Agentbench: Evaluating llms as agents," in *International Conference on Learning Representations (ICLR)*, 2024.

[5] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," *arXiv preprint arXiv:2312.00752*, 2023.

[6] T. Dao and A. Gu, "Transformers are ssms: Generalized models and efficient algorithms through structured state space duality," *arXiv preprint arXiv:2405.21060*, 2024.

[7] NVIDIA, "Nemotron-h: A family of accurate and efficient hybrid mamba-transformer models," 2025.

[8] X. Dong, Y. Fu, S. Diao *et al.*, "Hymba: A hybrid-head architecture for small language models," 2024.

[9] H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[10] B. Li, Y. Zhang, D. Guo, R. Zhang, F. Li, H. Zhang, K. Zhang, P. Zhang, Y. Li, Z. Liu, and C. Li, "Llava-onevision: Easy visual task transfer," 2024.

[11] M. Liu, T. E. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu *et al.*, "Chipnemo: Domain-adapted llms for chip design," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] J. Blocklove, R. Chapman, B. Tan, D. Chen *et al.*, "Chipbench: The llm chip design benchmark," 2024.

[13] M. Shoeybi, M. Patwary, R. Puri, P. LeGresley, J. Casper, and B. Catanzaro, "Megatron-lm: Training multi-billion parameter language models using model parallelism," 2020.

[14] Y. Liu, S. Hou, X. Chen, Y. Wang, S. Yuan, Q. Gan, S. You, F. Chen, W. Li, S. Hu, Y. Liu, E. Xia, X. Lu, H. Wang, F. Xu, and Y. Li, "Zhulong: Execution-grounded llm agent for eda scripting with offline api self-exploration," 2026.

[15] A. El-Nouby, J. a. Tremblay, V. Bansal, V. Sharma, A. Pumarola, I. Starukhin, A. Po, O. Klein, J. Shlens, and J. Susskind, "Aimv2: A scalable and data-efficient pre-training pipeline," 2024. [Online]. Available: https://arxiv.org/abs/2411.14402

[16] M. Tschannen, A. Pavlov, G. Lermen, A. El-Nouby, A. Kumar, R. Wightman, L. Beyer *et al.*, "Siglip 2: Multilingual vision-language encoders with improved semantic understanding," 2025. [Online]. Available: https://arxiv.org/abs/2502.14786

[17] Z.-V. Li, W.-N. Lin, P. Potharaju, P. Bhattacharya, K. Pandey, Y. Shao, Y. Wang, X. Li, Y. Serenko, W.-C. F. Ma, J. Jiang, J. Zhao *et al.*, "Openvision2: A simple recipe for scaling up vision pretraining," 2025. [Online]. Available: https://arxiv.org/abs/2509.01644

[18] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022. [Online]. Available: https://arxiv.org/abs/2111.06377

[19] C. Fan, G. Liu, M. Hong, R. R. Kompella, and S. Liu, "Rethinking muon beyond pretraining: Spectral failures and high-pass remedies for vla and rlvr," 2026. [Online]. Available: https://arxiv.org/abs/2605.19282